

Module 2

IEEE 1364-2001 (Verilog-2001)

Learning objectives

- Master the major Verilog update (IEEE 1364-2001) and its additions over 1364-1995: ANSI ports, impl...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["IEEE 1364-2001 Context"]

T2["ANSI-Style Port Declarations"]

T1 --> T2

T3["Implicit Sensitivity: always"]

T2 --> T3

Master the major Verilog update (IEEE 1364-2001) and its additions over 1364-1995: ANSI ports, implicit sensitivity, generate, signed types...

Overview

- This module covers IEEE Std 1364-2001 (Verilog-2001), the first major revision of the Verilog stand...

Syllabus topics

1. IEEE 1364-2001 Context (1/2)

- ANSI-style port declarations: Direction and type (and size) in the port list
- `**`always @*`**`: Implicit sensitivity for combinational logic (no manual list)
- `generate`: Conditional and loop-based instantiation and logic
- `signed`: Signed arithmetic and signed nets/variables
- Multi-dimensional arrays: e.g. ``reg [7:0] mem [0:255]``
- `localparam`: Named constants that cannot be overridden

1. IEEE 1364-2001 Context (2/2)

- ANSI-style task/function: Input/output in the header
- Other: File I/O improvements, ``$signed``/``$unsigned``, attribute syntax, etc.
- No SystemVerilog: no ``logic``, ``always_comb``/``always_ff``, interfaces, packages, classes
- No type parameters or interfaces (those are 1800)

2. ANSI-Style Port Declarations (2001)

- input wire: Default for input; can omit ``wire`` (e.g. ``input [7:0] a``).
- output reg: Required when the output is driven in ``always`/`initial``.
- output wire: When the output is driven only by ``assign`` or submodule.
- Same behavior; 2001 is more concise and keeps port and type in one place.

3. Implicit Sensitivity: always @* (2001)

- `**`@*`` (or ``@(*)`**`): Sensitive to every signal read in the block (RHS and conditions).
- Use for combinational logic only; do not use for sequential (use ``always @(posedge clk)``).
- `**Prefer `@*`**`: For combinational blocks; fewer errors, easier to maintain.
- Explicit list: When targeting strict 1995 compatibility or when a tool does not support ``@*``.

4. Generate (2001)

- genvar: Loop variable for generate loops; not a runtime variable.
- generate for: Replicates logic/instances; index must be constant at elaboration.
- generate if / else: Choose one of several implementations at elaboration.

5. Signed Types (2001)

- signed: Affects arithmetic and comparison in expressions involving that net/reg.
- Unsigned (default): No sign extension; values 0 to 2^N-1 .

6. Multi-Dimensional Arrays (2001)

- Packed (left): `[7:0]` — contiguous bits, can be used as a single vector.
- Unpacked (right): `[0:255]` — array of elements; select with one index at a time in 2001.

7. localparam (2001)

- parameter: Can be overridden: ``#(.WIDTH(16))``.
- localparam: Cannot be overridden; often derived from parameters or fixed.

8. ANSI-Style Tasks and Functions (2001)

- automatic: Optional; gives automatic storage (re-entrant). Useful for recursive or concurrent calls.
- input/output in header: Cleaner than 1995 style; avoids duplicate declarations.
- Note: In 1364-2001 use ``output reg`` (not ``output logic``; ``logic`` is SystemVerilog).

Hands-on examples

Module 2 toolchain check

```
$ command -v iverilog >/dev/null && echo "Toolchain ready: $(iverilog  
-V 2>&1 | head -1)"
```

Terminal: module2_check

Toolchain ready: Icarus Verilog version 12.0 (stable) ()

Demo: ANSI Ports

```
$ cd module2/examples/ansi_ports && make clean && make run
```

Terminal: ex01_ansi_ports

```
rm -f top
iverilog -o top mux2.v
vvp top
ANSI ports (1364-2001): direction and type in port list
sel a b | y
0 0 0 | 0
0 0 1 | 0
0 1 0 | 1
0 1 1 | 1
1 0 0 | 0
1 0 1 | 1
1 1 0 | 0
1 1 1 | 1
mux2.v:34: $finish called at 80 (1s)
```

Demo: always @*

```
$ cd module2/examples/procedural && make clean && make run
```

Terminal: ex02_procedural

```
rm -f top
iverilog -o top always_star.v
vvp top
always @* (1364-2001): implicit sensitivity
a b c sel | y_comb mux_out
0 0 0 0 | 0 0
1 1 0 0 | 1 1
0 1 0 1 | 0 1
always_star.v:37: $finish called at 30 (1s)
```

Demo: Generate

```
$ cd module2/examples/generate && make clean && make run
```

```
Terminal: ex03_generate  
rm -f top  
iverilog -o top param_mux.v  
vvp top  
Generate (1364-2001): parameterized mux  
  sel=0 a=1010 b=0101 y=1010  
  sel=1 a=1010 b=0101 y=0101  
param_mux.v:43: $finish called at 20 (1s)
```

Demo: Generate if/else

```
$ cd module2/examples/generate_if && make clean && make run
```

Terminal: ex04_generate_if

```
rm -f top
iverilog -o top gen_if.v
vvp top
generate if/else (1364-2001): USE_CLIP selects implementation
  a=10 b= 5 wrap=15 clip=15
  a=12 b= 8 wrap= 4 clip=15
gen_if.v:34: $finish called at 20 (1s)
```

Demo: Generate Ripple Adder

```
$ cd module2/examples/generate_ripple_adder && make clean && make run
```

```
Terminal: ex05_generate_ripple_adder  
rm -f top  
iverilog -o top ripple_adder.v  
vvp top  
generate for: N-bit ripple-carry adder (1364-2001)  
  3 + 5 + 0 = 8 cout=0  
 15 + 1 + 0 = 0 cout=1  
 7 + 7 + 1 = 15 cout=0  
ripple_adder.v:55: $finish called at 30 (1s)
```

Demo: Signed

```
$ cd module2/examples/signed && make clean && make run
```

Terminal: ex06_signed

```
rm -f top
iverilog -o top adder_signed.v
vvp top
Signed (1364-2001): signed reg/wire, $signed/$unsigned
  a= 50 b= 30 sum_signed= 80 sum_unsigned= 80
  a= -10 b= 20 sum_signed= 10 sum_unsigned=266
  a=-100 b= -30 sum_signed=-130 sum_unsigned=382
adder_signed.v:28: $finish called at 30 (1s)
```

Demo: Signed vs Unsigned Comparison

```
$ cd module2/examples/signed_compare && make clean && make run
```

Terminal: ex07_signed_compare

```
rm -f top
iverilog -o top signed_compare.v
vvp top
Signed vs unsigned comparison (1364-2001): $signed() in expressions
  a=200 b= 50  lt_unsigned=0 lt_signed=1
  a=0xf0 b= 10  lt_unsigned=0 lt_signed=1
signed_compare.v:28: $finish called at 20 (1s)
```


Demo: Decoder

```
$ cd module2/examples/decoder && make clean && make run
```

Terminal: ex08_decoder

```
rm -f top
iverilog -o top decoder.v
vvp top
Decoder 2:4 (1364-2001): ANSI ports, always @*
sel | y
0 | 0001
1 | 0010
2 | 0100
3 | 1000
decoder.v:33: $finish called at 40 (1s)
```

Demo: Multi-Dimensional Arrays

```
$ cd module2/examples/arrays && make clean && make run
```

Terminal: ex09_arrays

```
rm -f top
iverilog -o top ram_simple.v
vvp top
Multi-dimensional arrays (1364-2001): reg [7:0] mem [0:255]
  addr=10 dout=a5
  addr=20 dout=b2
ram_simple.v:54: $finish called at 86 (1s)
```

Demo: Multi-Dim Style

```
$ cd module2/examples/multi_dim_arrays && make clean && make run
```

Terminal: ex10_multi_dim_arrays

```
rm -f top
iverilog -o top multi_dim.v
vvp top
Multi-dimensional style (1364-2001): buf[row*4+col] for 2x4
[0][1]=a1
[1][2]=b2
multi_dim.v:59: $finish called at 86 (1s)
```

Demo: Parameters and localparam

```
$ cd module2/examples/parameters && make clean && make run
```

Terminal: ex11_parameters

```
rm -f top
iverilog -o top counter_param.v
vvp top
parameter and localparam (1364-2001): WIDTH=4, MAX=15
count= 0
count= 1
count= 2
count= 3
count= 4
count= 5
count= 6
count= 7
count= 8
count= 9
count=10
count=11
count=12
count=13
count=14
count=15
count= 0
count= 1
count= 2
count= 3
counter_param.v:37: $finish called at 235 (1s)
```

Demo: ANSI Task/Function

```
$ cd module2/examples/tasks_functions && make clean && make run
```

Terminal: ex12_tasks_functions

```
rm -f top
iverilog -o top ansi_task_func.v
vvp top
ANSI task/function (1364-2001): input/output in header
  x= 10 y= 20 add8= 30 prod(low8)=200
  x=  5 y=  6 add8= 11 prod(low8)= 30
ansi_task_func.v:38: $finish called at 20 (1s)
```

Demo: ANSI Task with Output

```
$ cd module2/examples/task_ansi && make clean && make run
```

Terminal: ex13_task_ansi

```
rm -f top
iverilog -o top task_ansi.v
vvp top
ANSI task with output (1364-2001): task apply_reset(output reg rn)
t=125 rst_n=1 count= x
t=135 rst_n=1 count= x
t=145 rst_n=1 count= x
t=155 rst_n=1 count= x
t=165 rst_n=1 count= x
task_ansi.v:33: $finish called at 165 (1s)
```

Practice & assessment

Exercises

- ANSI Ports
- always @
- Generate
- Signed
- Arrays and localparam

Exercises

- Using always @* for Sequential Logic
- genvar in Normal Code
- Overriding localparam
- Signed vs Unsigned Mix
- Generate Block Scope

Summary & next steps

- Master the major Verilog update (IEEE 1364-2001) and its additions over 1364-1995: ANSI ports, impl...
- Complete module2/CHECKLIST.md
- Run `./scripts/module2.sh --check`
- Next: Next module in course